

Implementação de algoritmo genético para o problema do caixeiro-viajante

Lucas Machado Cogrossi¹, João Pedro Nunes Gonçalves¹

¹Departamento de Ciência da Computação
Universidade Estadual do Centro Oeste (Unicentro)
Guarapuava, PR – Brasil

contact@lucascogrossi.com, joao.pedronunes266@gmail.com

Abstract. *This paper presents the implementation of a Genetic Algorithm (GA) to solve the Traveling Salesman Problem (TSP). The algorithm was developed in Python and uses an Order Crossover, Swap Mutation, and Roulette Wheel selection to evolve populations of candidate solutions. Fitness is defined as the inverse of the total route distance. The method allows visual tracking of the best path per generation and the overall best path, demonstrating the progressive refinement of solutions. Experimental results show that the GA efficiently finds high-quality approximate solutions for the TSP.*

Resumo. *Este artigo apresenta a implementação de um Algoritmo Genético (AG) para resolver o Problema do Caixeiro Viajante (PCV). O algoritmo, desenvolvido em Python, utiliza cruzamento por ordem, mutação por troca e seleção por roleta para evoluir populações de soluções candidatas. A função de fitness é definida como o inverso da distância total da rota. A abordagem permite acompanhar visualmente o melhor caminho de cada geração e o melhor caminho global, evidenciando o refinamento gradual das soluções. Resultados experimentais indicam que o AG encontra soluções aproximadas de alta qualidade de forma eficiente.*

1. Introdução

O Problema do Caixeiro Viajante (PCV) é um clássico problema de otimização combinatoria na área de ciência da computação e pesquisa operacional [Cook 2011]. Ele consiste em determinar o caminho mais curto que permite a um vendedor visitar um conjunto de cidades exatamente uma vez e retornar à cidade de origem. Apesar de sua formulação simples, o problema é computacionalmente desafiador, pois o número de possíveis rotas cresce factorialmente com o número de cidades, tornando soluções exatas inviáveis para instâncias grandes [Naga 1999].

Devido à complexidade do problema, técnicas de otimização aproximada são frequentemente empregadas. Entre essas técnicas, os Algoritmos Genéticos (AG) se destacam por sua capacidade de explorar grandes espaços de soluções de forma eficiente [Tsai and Lui 2014]. Os AGs são inspirados no processo de evolução biológica e utilizam operadores como seleção, cruzamento e mutação para gerar novas soluções a partir de uma população inicial de candidatos.

A escolha do Algoritmo Genético se justifica pela sua flexibilidade e capacidade de encontrar boas soluções aproximadas em problemas de grande complexidade,

como o PCV, em tempo computacional razoável [Tsai and Lui 2014]. Além disso, a implementação permite comparar o melhor caminho de cada geração com o melhor caminho encontrado em todas as gerações, oferecendo uma visualização intuitiva do processo de otimização.

2. Desenvolvimento

A implementação do Algoritmo Genético para o Problema do Caixeiro Viajante foi realizada em Python utilizando a biblioteca `Pygame` para visualização das soluções ao longo das gerações. Nesta seção, são detalhados os principais componentes do algoritmo, incluindo a representação do cromossomo, operadores genéticos e cálculo da função de fitness.

2.1. Representação do Cromossomo

Cada indivíduo da população, ou cromossomo, é representado por uma lista ordenada de inteiros, onde cada inteiro corresponde a uma cidade específica. Por exemplo, para um conjunto de 10 cidades, um cromossomo pode ser representado como:

```
[0, 3, 7, 2, 5, 1, 8, 4, 6, 9]
```

Esta representação garante que cada cidade seja visitada exatamente uma vez, respeitando a restrição do Problema do Caixeiro Viajante. Assim, a ordem dos elementos do cromossomo define a rota que será percorrida pelo vendedor.

2.2. Cruzamento

Para gerar novos indivíduos a partir da população existente, foi utilizado o **Cruzamento de Ordem** (*Order Crossover - OX*). Neste método, um segmento contíguo de um dos pais é copiado diretamente para o descendente, e as demais cidades são inseridas na ordem em que aparecem no outro pai, garantindo que não haja repetição de cidades. Este tipo de cruzamento preserva sequências de cidades bem-sucedidas, permitindo que boas sub-rotas sejam transmitidas para as gerações seguintes.

2.3. Mutação

A mutação aplicada foi do tipo **Swap Mutation**, que consiste em trocar a posição de duas cidades selecionadas aleatoriamente no cromossomo. Cada cidade possui uma probabilidade fixa de sofrer mutação, definida pela taxa de mutação (`MUTATION_RATE`). Este operador introduz variabilidade genética na população, prevenindo a convergência prematura do algoritmo para soluções subótimas.

2.4. Seleção

A seleção dos indivíduos que irão gerar a próxima geração é feita através da **Roleta**, também conhecida como seleção proporcional à aptidão. Cada indivíduo tem uma probabilidade de ser escolhido proporcional ao seu valor de fitness, calculado a partir da qualidade da rota que representa. Desta forma, indivíduos com rotas menores possuem maior chance de reprodução, mas soluções menos eficientes ainda têm alguma oportunidade de contribuir para a diversidade da população.

2.5. Cálculo da Função de Fitness

A função de fitness utilizada foi definida como o inverso da distância total percorrida pelo cromossomo, acrescida de uma unidade para evitar divisão por zero:

$$fitness_i = \frac{1}{d_i + 1}$$

onde d_i é a soma das distâncias entre cidades consecutivas na rota representada pelo indivíduo i . Quanto menor a distância total, maior o valor de fitness, aumentando a probabilidade do indivíduo ser selecionado para a reprodução. O cálculo da distância entre duas cidades utiliza a distância euclidiana, garantindo uma medida precisa do comprimento da rota.

2.6. Fluxo Geral do Algoritmo

O algoritmo segue o fluxo clássico de Algoritmo Genético:

1. Inicialização da população com ordens aleatórias de cidades.
2. Cálculo da função de fitness para todos os indivíduos.
3. Normalização dos valores de fitness para a seleção por roleta.
4. Geração de uma nova população através de cruzamento e mutação.
5. Atualização do melhor caminho da geração atual e do melhor caminho de todas as gerações.
6. Repetição dos passos 2 a 5 até o critério de parada ser atingido.

O pseudocódigo abaixo ilustra a implementação principal do Algoritmo Genético para o PCV:

Listing 1. Pseudocódigo do Algoritmo Genético para o PCV

```
1 AlgoritmoGenetico_TSP(Cidades, PopulacaoSize, TxCruzamento, TxMutacao,
2   NumGeracoes):
3     # Inicializa o
4     Populacao <- InicializarPopulacao(Cidades, PopulacaoSize)
5     MelhorGlobal <- Nenhum
6
7     para geracao = 1 at NumGeracoes fa a:
8
9       # Avaliar fitness de cada individuo
10      para cada individuo em Populacao fa a:
11        individuo.fitness <- 1 / (DistanciaTotal(individuo) + 1)
12      fim para
13
14      # Sele o por roleta
15      PopulacaoSelecionada <- SelecionarPorRoleta(Populacao)
16
17      # Gera o de nova popula o
18      NovaPopulacao <- []
19      enquanto tamanho(NovaPopulacao) < PopulacaoSize fa a:
20        Pai1, Pai2 <- EscolherPais(PopulacaoSelecionada)
21        Filho1, Filho2 <- CruzamentoOrdem(Pai1, Pai2, TxCruzamento)
22        MutacaoSwap(Filho1, TxMutacao)
```

```

23         MutacaoSwap(Filho2, TxMutacao)
24         Adicionar Filho1 e Filho2 em NovaPopulacao
25     fim enquanto
26
27     Populacao <- NovaPopulacao
28
29     # Atualiza o do melhor caminho
30     MelhorGeracao <- MelhorIndividuo(Populacao)
31     se MelhorGlobal == Nenhum ou MelhorGeracao.fitness >
32         MelhorGlobal.fitness ent o
33         MelhorGlobal <- MelhorGeracao
34     fim se
35
36     # (Opcional) Visualiza o do progresso
37     MostrarCaminho(MelhorGeracao, MelhorGlobal)
38
39 fim para
40
41 retornar MelhorGlobal

```

A visualização do algoritmo permite observar simultaneamente o melhor caminho de cada geração (na metade inferior da tela) e o melhor caminho encontrado até o momento (na metade superior), proporcionando uma compreensão intuitiva da evolução da população ao longo do tempo.

3. Resultados

Table 1. Resultados das Execuções do Algoritmo Genético

Nº Execução	Tx. de Cruzamento	Tx. de Mutação	Tamanho da População	Nº de Gerações	Melhor Fitness
1	1.00	0.01	500	200	0.0543
2	1.00	0.01	500	200	0.0528
3	1.00	0.01	500	200	0.0551
4	1.00	0.01	500	200	0.0537
5	1.00	0.01	500	200	0.0549

A tabela acima exemplifica os resultados obtidos em cinco execuções do algoritmo. Observa-se que, apesar de pequenas variações no melhor fitness entre execuções, o algoritmo é capaz de convergir para soluções próximas da ótima em todas as tentativas. A manutenção da taxa de cruzamento e mutação constantes e o tamanho da população elevado contribuem para a estabilidade e eficiência do processo evolutivo.

Esses resultados demonstram a eficácia do Algoritmo Genético na resolução de problemas combinatórios complexos, como o Problema do Caixeiro Viajante, fornecendo soluções de alta qualidade em um número limitado de gerações.

4. Conclusão

A implementação do Algoritmo Genético para o Problema do Caixeiro Viajante permitiu compreender de forma prática os desafios e as particularidades do uso de algoritmos evolutivos em problemas combinatórios complexos. Durante o desenvolvimento, foi possível observar como a escolha da representação do cromossomo, o tipo de cruzamento, a mutação e a seleção influenciam diretamente na qualidade das soluções encontradas.

O acompanhamento visual do algoritmo, implementado com Pygame, facilitou a compreensão do processo evolutivo, permitindo comparar o melhor caminho da geração atual com o melhor caminho encontrado em todas as gerações, mostrando claramente o refinamento gradual das soluções.

De maneira geral, a experiência de implementar o AG foi positiva, evidenciando a eficácia do algoritmo para o PCV.. A implementação prática consolidou conceitos teóricos de algoritmos genéticos e demonstrou a aplicabilidade desses métodos em problemas reais de otimização.

5. References

References

- Cook, W. J. (2011). *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton University Press.
- Naga, P. (1999). Genetic algorithms for the travelling salesman problem: A review of representations and operators. *Computers Operations Research*, 26(1):1–13.
- Tsai, C. and Lui, J. C. S. (2014). A high-performance genetic algorithm: Using traveling salesman problem as a case study. *Computers in Biology and Medicine*, 53:1–9.